

Algoritmos_AG1

September 17, 2025

#Algoritmos - Actividad Guiada 1

Nombre: Alejandro Napoleon Thoux URL: [https://colab.research.google.com/github/catilo/algoritmos_optimizacion](https://colab.research.google.com/github/catilo/algoritmos_optimizacion/blob/main/actividades/Algoritmos_AG1.ipynb)

[http:https://github.com/catilo/algoritmos_optimizacion/blob/main/actividades/Algoritmos_AG1.ipynb](https://github.com/catilo/algoritmos_optimizacion/blob/main/actividades/Algoritmos_AG1.ipynb)

##Torres de Hanoi con Divide y vencerás

```
[2]: def Torres_Hanoi(N, desde, hasta):  
    if N ==1 :  
        print("Lleva la ficha " ,desde , " hasta " , hasta )  
  
    else:  
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        print("Lleva la ficha " ,desde , " hasta " , hasta )  
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )  
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )  
  
Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha 1 hasta 3  
Lleva la ficha 1 hasta 2  
Lleva la ficha 3 hasta 2  
Lleva la ficha 1 hasta 3  
Lleva la ficha 2 hasta 1  
Lleva la ficha 2 hasta 3  
Lleva la ficha 1 hasta 3
```

```
[14]: #Sucesión de Fibonacci  
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci  
#Calculo del termino n-simo de la sucesión de Fibonacci  
def Fibonacci(N:int):  
    if N < 2:  
        return 1  
    else:  
        return Fibonacci(N-1)+Fibonacci(N-2)
```

```
Fibonacci(5)
```

```
[14]: 8
```

```
[39]: print(Fibonacci(40))
```

```
165580141
```

```
[40]: import cProfile
```

```
cProfile.run("Fibonacci(40)", sort='ncalls')
```

```
331160977 function calls (698 primitive calls) in 52.075 seconds
```

```
Ordered by: call count
```

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
331160275/0	52.071	0.000	0.000		3743385582.py:4(Fibonacci)
88/84	0.000	0.000	0.000	0.000	{built-in method
builtins.isinstance}					
39	0.000	0.000	0.000	0.000	enum.py:1587(_get_value)
28	0.000	0.000	0.000	0.000	{built-in method builtins.len}
22	0.000	0.000	0.000	0.000	base_events.py:766(time)
22	0.000	0.000	0.000	0.000	{built-in method time.monotonic}
21	0.000	0.000	0.000	0.000	base_events.py:2052(get_debug)
17	0.000	0.000	0.000	0.000	enum.py:695(__call__)
17	0.000	0.000	0.000	0.000	enum.py:1154(__new__)
15	0.000	0.000	0.000	0.000	threading.py:605(is_set)
15	0.000	0.000	0.000	0.000	threading.py:1134(is_alive)
15	0.000	0.000	0.000	0.000	{method 'is_done' of
'_thread._ThreadHandle' objects}					
13	0.000	0.000	0.000	0.000	typing.py:2371(cast)
12	0.000	0.000	0.000	0.000	{method 'append' of
'collections.deque' objects}					
12	0.000	0.000	0.000	0.000	{method 'popleft' of
'collections.deque' objects}					
12	0.000	0.000	0.000	0.000	{built-in method builtins.max}
11	0.000	0.000	0.000	0.000	
selector_events.py:740(_process_events)					
11	0.000	0.000	0.001	0.000	events.py:87(_run)
11	0.000	0.000	0.002	0.000	base_events.py:1954(_run_once)
11	0.000	0.000	0.001	0.000	selectors.py:310(select)
11	0.000	0.000	0.000	0.000	base_events.py:548(_check_closed)
11	0.000	0.000	0.001	0.000	selectors.py:304(_select)
11	0.000	0.000	0.000	0.000	events.py:36(__init__)
11	0.000	0.000	0.001	0.000	{method 'run' of
'_contextvars.Context' objects}					
11	0.000	0.000	0.001	0.000	{built-in method select.select}

10	0.000	0.000	0.000	0.000	tasks.py:703(sleep)
9	0.000	0.000	0.000	0.000	{method '__exit__' of
'_thread.lock' objects}					
8	0.000	0.000	0.000	0.000	enum.py:1594(__or__)
8	0.000	0.000	0.000	0.000	{built-in method builtins.hasattr}
7	0.000	0.000	0.000	0.000	socket.py:623(send)
6	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
6	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
6	0.000	0.000	0.000	0.000	{built-in method
_contextvars.copy_context}					
6	0.000	0.000	0.000	0.000	{built-in method
_asyncio.get_running_loop}					
5	0.000	0.000	0.000	0.000	enum.py:1605(__and__)
5	0.000	0.000	0.000	0.000	base_events.py:799(call_at)
5	0.000	0.000	0.000	0.000	base_events.py:775(call_later)
5	0.000	0.000	0.000	0.000	
base_events.py:1949(_timer_handle_cancelled)					
5	0.000	0.000	0.000	0.000	base_events.py:457(create_future)
5	0.000	0.000	0.000	0.000	events.py:113(__init__)
5	0.000	0.000	0.000	0.000	events.py:157(cancel)
5	0.000	0.000	0.000	0.000	events.py:73(cancel)
5	0.000	0.000	0.000	0.000	
futures.py:310(_set_result_unless_cancelled)					
5	0.000	0.000	0.000	0.000	iostream.py:127(_event_pipe_gc)
5	0.000	0.000	0.001	0.000	
iostream.py:118(_run_event_pipe_gc)					
5	0.000	0.000	0.000	0.000	{built-in method _heapq.heappush}
5	0.000	0.000	0.000	0.000	{built-in method _heapq.heappop}
5	0.000	0.000	0.000	0.000	{built-in method math.isnan}
5	0.000	0.000	0.000	0.000	{method 'items' of 'dict' objects}
5	0.000	0.000	0.000	0.000	{method 'set_result' of
'_asyncio.Future' objects}					
5	0.000	0.000	0.000	0.000	{method 'cancelled' of
'_asyncio.Future' objects}					
4	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
3	0.000	0.000	0.000	0.000	<frozen
importlib._bootstrap>:1390(_handle_fromlist)					
3	0.000	0.000	0.000	0.000	typing.py:426(inner)
3	0.000	0.000	0.000	0.000	attrsettr.py:43(__getattr__)
3	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
3	0.000	0.000	0.000	0.000	zmqstream.py:528(receiving)
3	0.000	0.000	0.000	0.000	{method 'upper' of 'str' objects}
3	0.000	0.000	0.000	0.000	{built-in method builtins.getattr}
2	0.000	0.000	0.000	0.000	<frozen
abc>:121(__subclasscheck__)					
2	0.000	0.000	0.000	0.000	typing.py:1443(__hash__)
2	0.000	0.000	0.000	0.000	typing.py:1374(__instancecheck__)
2	0.000	0.000	0.000	0.000	typing.py:1665(__subclasscheck__)

2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
2	0.000	0.000	0.000	0.000	
traitlets.py:3486(validate_elements)					
2	0.000	0.000	0.000	0.000	queue.py:115(empty)
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
2	0.000	0.000	0.000	0.000	
traitlets.py:3624(validate_elements)					
2	0.000	0.000	0.000	0.000	queue.py:267(_qsize)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
2	0.000	0.000	0.000	0.000	
zmqstream.py:653(_rebuild_io_state)					
2	0.000	0.000	0.000	0.000	{built-in method
_abc._abc_subclasscheck}					
2	0.000	0.000	0.000	0.000	{method 'acquire' of
'_thread.lock' objects}					
2	0.000	0.000	0.000	0.000	{method 'extend' of 'list'
objects}					
2	0.000	0.000	0.000	0.000	{built-in method builtins.hash}
2	0.000	0.000	0.000	0.000	{built-in method
builtins.issubclass}					
2	0.000	0.000	0.000	0.000	{method '__exit__' of
'sqlite3.Connection' objects}					
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
1	0.000	0.000	0.000	0.000	threading.py:318(_is_owned)
1	0.000	0.000	0.000	0.000	threading.py:303(__enter__)
1	0.000	0.000	0.000	0.000	threading.py:312(_release_save)
1	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
1	0.000	0.000	0.000	0.000	socket.py:700(send_multipart)
1	0.000	0.000	0.000	0.000	zmqstream.py:573(_handle_events)
1	0.000	0.000	0.000	0.000	traitlets.py:629(get)
1	0.000	0.000	0.000	0.000	
traitlets.py:1527(_notify_observers)					
1	0.000	0.000	0.000	0.000	zmqstream.py:614(_handle_recv)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
1	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
1	0.000	0.000	0.000	0.000	zmqstream.py:546(_run_callback)
1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
1	0.000	0.000	0.000	0.000	iostream.py:278(_really_send)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
1	0.000	0.000	0.000	0.000	

```

history.py:1016(_writeout_output_cache)
    1    0.000    0.000    0.000    0.000 history.py:1065(hold)
    1    0.001    0.001    0.001    0.001 iostream.py:276(<lambda>)
    1    0.000    0.000    0.000    0.000 iostream.py:255(closed)
    1    0.000    0.000    0.000    0.000 iostream.py:216(_check_mp_mode)
    1    0.000    0.000    0.000    0.000
iostream.py:213(_is_master_process)
    1    0.000    0.000    0.000    0.000 iostream.py:157(_handle_event)
    1    0.000    0.000    0.000    0.000 {built-in method nt.getpid}
    1    0.000    0.000    0.000    0.000 {method 'release' of
'_thread.lock' objects}
    1    0.000    0.000    0.000    0.000 {method '__enter__' of
'_thread.lock' objects}
    1    0.000    0.000    0.000    0.000 {built-in method
_thread.allocate_lock}
    1    0.000    0.000    0.000    0.000 {built-in method builtins.next}
    1    0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}

```

Sucesión de Fibonacci usando la Relación de Oro (

$\varphi = \frac{1+\sqrt{5}}{2} = 1.6180339887\dots$) implementando la formula de Binet. Este método pierde precisión mas alla de $F(n) = 34$ debido al error en el calculo por el uso de aritmetica de punto flotante. Ademas, en los primeros 5 números de la serie, el resultado no es preciso. La complejidad es de $O(n)$.

```

[30]: def fibGoldenRatio(n: int):
        """
        GR = 1.6180339887
        fib = [1, 1, 2, 3, 5, 8 ]

        if n < 6:
            return fib[n]

        termino = 5
        fibonacci = 8

        while termino < n:
            fibonacci = round(fibonacci * GR)
            termino+=1

        return fibonacci

```

```

[42]: print(fibGoldenRatio(40))

```

165580141

```
[41]: import cProfile

cProfile.run("fibGoldenRatio(40)", sort='ncalls')
```

39 function calls in 0.000 seconds

Ordered by: call count

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
35	0.000	0.000	0.000	0.000	{built-in method builtins.round}
1	0.000	0.000	0.000	0.000	<string>:1(<module>)
1	0.000	0.000	0.000	0.000	2927278474.py:1(fibGoldenRatio)
1	0.000	0.000	0.000	0.000	{built-in method builtins.exec}
1	0.000	0.000	0.000	0.000	{method 'disable' of

'_lsprof.Profiler' objects}

##Devolución de cambio por técnica voraz

```
[60]: def cambio_monedas(N, SM):
    SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
    ValorAcumulado = 0

    for i,valor in enumerate(SM):
        monedas = (N-ValorAcumulado)//valor
        SOLUCION[i] = monedas
        ValorAcumulado = ValorAcumulado + monedas*valor

    if ValorAcumulado == N:
        return SOLUCION

cambio_monedas(15,[25,10,5,1])
```

[60]: [0, 1, 1, 0]

```
[61]: import cProfile

cProfile.run("cambio_monedas(15,[25,10,5,1])", sort='ncalls')
```

5 function calls in 0.000 seconds

Ordered by: call count

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.000	0.000	514988140.py:1(cambio_monedas)
1	0.000	0.000	0.000	0.000	<string>:1(<module>)
1	0.000	0.000	0.000	0.000	{built-in method builtins.exec}

```

1      0.000      0.000      0.000      0.000 {built-in method builtins.len}
1      0.000      0.000      0.000      0.000 {method 'disable' of
'_lsprof.Profiler' objects}

```

Se agrega una condicion adicional para limitar la cantidad de monedas disponibles.

```

[63]: def cambio_monedas_extra(N, SM):
      SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
      ValorAcumulado = 0

      for i,valor in enumerate(SM):
          monedas = (N-ValorAcumulado)//valor[0]
          if monedas > valor[1]:
              monedas = valor[1]
              ValorAcumulado = ValorAcumulado + monedas*valor[0]
          else:
              ValorAcumulado = ValorAcumulado + monedas*valor[0]

          SOLUCION[i] = monedas

          if ValorAcumulado == N:
              return SOLUCION

      SOLUCION_SUB_OPTIMA = []
      [SOLUCION_SUB_OPTIMA.append(value[0]*value[1][0]) for value in
↪zip(SOLUCION,SM)]
      return f"No se encontro solucion optima: {SOLUCION} Suma total:
↪{sum(SOLUCION_SUB_OPTIMA)}"

cambio_monedas_extra(250,[(25,5),(10,1),(5,10),(1,23)])

```

```

[63]: 'No se encontro solucion optima: [5, 1, 10, 23] Suma total: 208'

```

```

[65]: import cProfile

cProfile.
↪run("cambio_monedas_extra(15,[(25,5),(10,1),(5,10),(1,23)])",sort='ncalls')

```

5 function calls in 0.000 seconds

Ordered by: call count

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.000	0.000	

```

1593940075.py:1(cambio_monedas_extra)
      1  0.000    0.000    0.000    0.000 <string>:1(<module>)
      1  0.000    0.000    0.000    0.000 {built-in method builtins.exec}
      1  0.000    0.000    0.000    0.000 {built-in method builtins.len}
      1  0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}

```

0.1 N-Reinas por técnica de vuelta atrás

```

[66]: def describe(S):
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X ", end="")
            else:
                print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la
    ↪misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.
        ↪count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

```



```

if es_prometedora(solucion, etapa):
    if etapa == N-1:
        print(solucion)
        #escribe(solucion)
        print()
    else:
        reinas(N, solucion, etapa+1)
else:
    None

solucion[etapa] = 0

reinas(8)

```

[1, 5, 8, 6, 3, 7, 2, 4]

[1, 6, 8, 3, 7, 4, 2, 5]

[1, 7, 4, 6, 8, 2, 5, 3]

[1, 7, 5, 8, 2, 4, 6, 3]

[2, 4, 6, 8, 3, 1, 7, 5]

[2, 5, 7, 1, 3, 8, 6, 4]

[2, 5, 7, 4, 1, 8, 6, 3]

[2, 6, 1, 7, 4, 8, 3, 5]

[2, 6, 8, 3, 1, 4, 7, 5]

[2, 7, 3, 6, 8, 5, 1, 4]

[2, 7, 5, 8, 1, 4, 6, 3]

[2, 8, 6, 1, 3, 5, 7, 4]

[3, 1, 7, 5, 8, 2, 4, 6]

[3, 5, 2, 8, 1, 7, 4, 6]

[3, 5, 2, 8, 6, 4, 7, 1]

[3, 5, 7, 1, 4, 2, 8, 6]

[3, 5, 8, 4, 1, 7, 2, 6]

[3, 6, 2, 5, 8, 1, 7, 4]
[3, 6, 2, 7, 1, 4, 8, 5]
[3, 6, 2, 7, 5, 1, 8, 4]
[3, 6, 4, 1, 8, 5, 7, 2]
[3, 6, 4, 2, 8, 5, 7, 1]
[3, 6, 8, 1, 4, 7, 5, 2]
[3, 6, 8, 1, 5, 7, 2, 4]
[3, 6, 8, 2, 4, 1, 7, 5]
[3, 7, 2, 8, 5, 1, 4, 6]
[3, 7, 2, 8, 6, 4, 1, 5]
[3, 8, 4, 7, 1, 6, 2, 5]
[4, 1, 5, 8, 2, 7, 3, 6]
[4, 1, 5, 8, 6, 3, 7, 2]
[4, 2, 5, 8, 6, 1, 3, 7]
[4, 2, 7, 3, 6, 8, 1, 5]
[4, 2, 7, 3, 6, 8, 5, 1]
[4, 2, 7, 5, 1, 8, 6, 3]
[4, 2, 8, 5, 7, 1, 3, 6]
[4, 2, 8, 6, 1, 3, 5, 7]
[4, 6, 1, 5, 2, 8, 3, 7]
[4, 6, 8, 2, 7, 1, 3, 5]
[4, 6, 8, 3, 1, 7, 5, 2]
[4, 7, 1, 8, 5, 2, 6, 3]
[4, 7, 3, 8, 2, 5, 1, 6]

[4, 7, 5, 2, 6, 1, 3, 8]
[4, 7, 5, 3, 1, 6, 8, 2]
[4, 8, 1, 3, 6, 2, 7, 5]
[4, 8, 1, 5, 7, 2, 6, 3]
[4, 8, 5, 3, 1, 7, 2, 6]
[5, 1, 4, 6, 8, 2, 7, 3]
[5, 1, 8, 4, 2, 7, 3, 6]
[5, 1, 8, 6, 3, 7, 2, 4]
[5, 2, 4, 6, 8, 3, 1, 7]
[5, 2, 4, 7, 3, 8, 6, 1]
[5, 2, 6, 1, 7, 4, 8, 3]
[5, 2, 8, 1, 4, 7, 3, 6]
[5, 3, 1, 6, 8, 2, 4, 7]
[5, 3, 1, 7, 2, 8, 6, 4]
[5, 3, 8, 4, 7, 1, 6, 2]
[5, 7, 1, 3, 8, 6, 4, 2]
[5, 7, 1, 4, 2, 8, 6, 3]
[5, 7, 2, 4, 8, 1, 3, 6]
[5, 7, 2, 6, 3, 1, 4, 8]
[5, 7, 2, 6, 3, 1, 8, 4]
[5, 7, 4, 1, 3, 8, 6, 2]
[5, 8, 4, 1, 3, 6, 2, 7]
[5, 8, 4, 1, 7, 2, 6, 3]
[6, 1, 5, 2, 8, 3, 7, 4]

[6, 2, 7, 1, 3, 5, 8, 4]
[6, 2, 7, 1, 4, 8, 5, 3]
[6, 3, 1, 7, 5, 8, 2, 4]
[6, 3, 1, 8, 4, 2, 7, 5]
[6, 3, 1, 8, 5, 2, 4, 7]
[6, 3, 5, 7, 1, 4, 2, 8]
[6, 3, 5, 8, 1, 4, 2, 7]
[6, 3, 7, 2, 4, 8, 1, 5]
[6, 3, 7, 2, 8, 5, 1, 4]
[6, 3, 7, 4, 1, 8, 2, 5]
[6, 4, 1, 5, 8, 2, 7, 3]
[6, 4, 2, 8, 5, 7, 1, 3]
[6, 4, 7, 1, 3, 5, 2, 8]
[6, 4, 7, 1, 8, 2, 5, 3]
[6, 8, 2, 4, 1, 7, 5, 3]
[7, 1, 3, 8, 6, 4, 2, 5]
[7, 2, 4, 1, 8, 5, 3, 6]
[7, 2, 6, 3, 1, 4, 8, 5]
[7, 3, 1, 6, 8, 5, 2, 4]
[7, 3, 8, 2, 5, 1, 6, 4]
[7, 4, 2, 5, 8, 1, 3, 6]
[7, 4, 2, 8, 6, 1, 3, 5]
[7, 5, 3, 1, 6, 8, 2, 4]
[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]

0.2 Viaje por el río. Programación dinámica

```
[67]: TARIFAS = [
    [0,5,4,3,999,999,999],
    [999,0,999,2,3,999,11],
    [999,999, 0,1,999,4,10],
    [999,999,999, 0,5,6,9],
    [999,999, 999,999,0,999,4],
    [999,999, 999,999,999,0,3],
    [999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
    #####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Iniciación de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(9999)*N]
    RUTA = [ [""]*N for i in range(9999)*N]

    for i in range(0,N-1):
        RUTA[i][i] = i                #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0             #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k                #Anota que para ir de i a j hay que pasar
↳por k
            PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####
```

```

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
            ',' + \
            str(RUTA[desde][hasta] \
                )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

```

PRECIOS
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

```

RUTA
[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']

```

La ruta es:

[67]: ',0,2,5'

[]: